

设计思路

结论

这套代码的核心思路是把机器人电控拆成五条清晰链路：输入、状态、控制、反馈、安全。只要这五条链路能讲清楚，就能证明代码不是拼凑出来的。

1. 为什么要模块化

如果把所有逻辑写在 `main.c`，短期看起来快，后期一定难维护。RoboMaster 电控不是一个函数能解决的问题，它至少包括：

- 遥控输入
- 底盘运动
- 云台姿态
- 电机闭环
- 安全保护
- 硬件接口

所以本工程拆成：

- `app.c`：任务调度
- `remote.c`：输入解析
- `chassis.c`：底盘控制
- `gimbal.c`：云台控制
- `pid.c`：闭环算法
- `safety.c`：安全状态
- `bsp_port.c`：硬件适配

2. 为什么硬件接口集中在 BSP 层

不同开发板的按键、LED、PWM、CAN、编码器、IMU 接法都不一样。如果控制算法直接调用 HAL，换板子就要大改算法层。

所以本工程规定：

- 算法层只调用 BSP_* 接口。
- 真实 HAL 代码只写在 bsp_port.c。
- 后续换硬件，只改 BSP，不改底盘和云台算法。

3. 为什么先做安全保护

机器人电控的底线不是“能跑”，而是“失控时能停”。因此本工程默认上电进入 STOP，并设置：

- 急停模式
- 输出限幅
- Pitch 角度限位
- 安全状态机

这样即使功能还不完整，也能保证调试风险可控。

4. 为什么底盘用速度分解

遥控器给的是车体速度 $v_x/v_y/w_z$ ，但电机需要的是四个轮子的目标速度。中间必须有运动学解算。

X 型四轮全向轮的基本思路是：前后、左右、旋转三个运动分量叠加到四个轮子上。这样机器人可以同时平移和旋转。

5. 为什么云台用 IMU 角度反馈

题目要求云台角度反馈不能用电机编码器代替。原因是云台最终要稳定的是姿态角，而不是电机轴转了多少。

所以设计中保留：

- BSP_GetYawAngleDeg()
- BSP_GetPitchAngleDeg()

真实上板时，这两个函数应从 IMU 获取角度。

Source: C:/Temp/codex_rm_assignment/rm_control_assignment_extracted.md:64